SecureGate

Course-Level Revision Plan and Structured System Report

CIS 5690 – Advanced Systems Project | Spring 2026

Executive Summary

SecureGate is a deployed full-stack digital access-control platform that replaces manual gate registers and shareable identity cards with cryptographically signed QR passes, role-based approval workflows, optional face verification, geofence-aware pass generation, and real-time audit logging. The deployed baseline already demonstrates the end-to-end system flow across four portals: Member, Admin, Guard, and Contact.

After the demo review, the project requires a stronger course-level framing. The feedback indicates three main issues:

- the use cases in the demo report are too implementation-centric and appear repetitive,
- GPS/geofence logic is repeated too often across the current write-up,
- the project needs one stronger scheduling or policy-driven feature to clearly reach course-project depth.

This revised report addresses those concerns by:

- restructuring the project around **12 distinct actor-goal use cases**,
- consolidating GPS into only two places: **instant on-campus pass generation** and **admin policy configuration**,
- introducing **slot-based access scheduling** as the primary course-level enhancement,
- clearly separating the **deployed baseline** from the **next implementation milestone**,
- presenting a realistic implementation roadmap, testing plan, and documentation plan.

Feedback-Driven Revision Strategy

Observed Demo Feedback	Revision Decision in This Report	Expected Impact
Use cases felt too similar	Rewrite the system around 12 actor-centered, goal-based use cases instead of feature-only descriptions	Stronger academic clarity and better problem-to-solution mapping
GPS appeared repeatedly in the report	Limit GPS to UC06 (instant on-campus pass) and UC11 (policy and boundary configuration)	Removes redundancy and makes the use-case set more distinct
Need stronger course-level depth	Add slot-based access scheduling with capacity, time-window, and policy enforcement	Adds a clear systems-design extension beyond the current deployment
Documentation overstated or mixed implemented and planned work	Split the report into deployed baseline, revised scope, and roadmap	More professional and defensible during evaluation
Need reasonable scope in short time	Prioritize one high-value extension plus analytics, testing, and demo alignment	Keeps the project ambitious but still finishable

Current Deployed Baseline

The currently deployed build already supports the following operational capabilities:

- access-request workflow with admin approval,
- JWT authentication and role-based access control,
- QR pass generation with HMAC-based integrity protection,
- standard pass request and approval flow,
- instant daily entry or exit pass generation,
- guard-side camera QR scanning and verification,
- optional face registration and gate-side face comparison,
- geofence-aware location validation,
- admin dashboard with pass queue and real-time logs,
- contact or parent history view through a signed access link,
- emergency exit workflow,
- Render deployment with Docker-based hosting.

The current baseline is therefore a valid working system, but it still needs a more rigorous project narrative and one stronger scheduling feature to meet course-level expectations cleanly.

Revised Course-Level Scope

Project objective: transform SecureGate from a functional gate-pass prototype into a policy-driven access-control system suitable for course evaluation in distributed systems, backend architecture, and access-governance design.

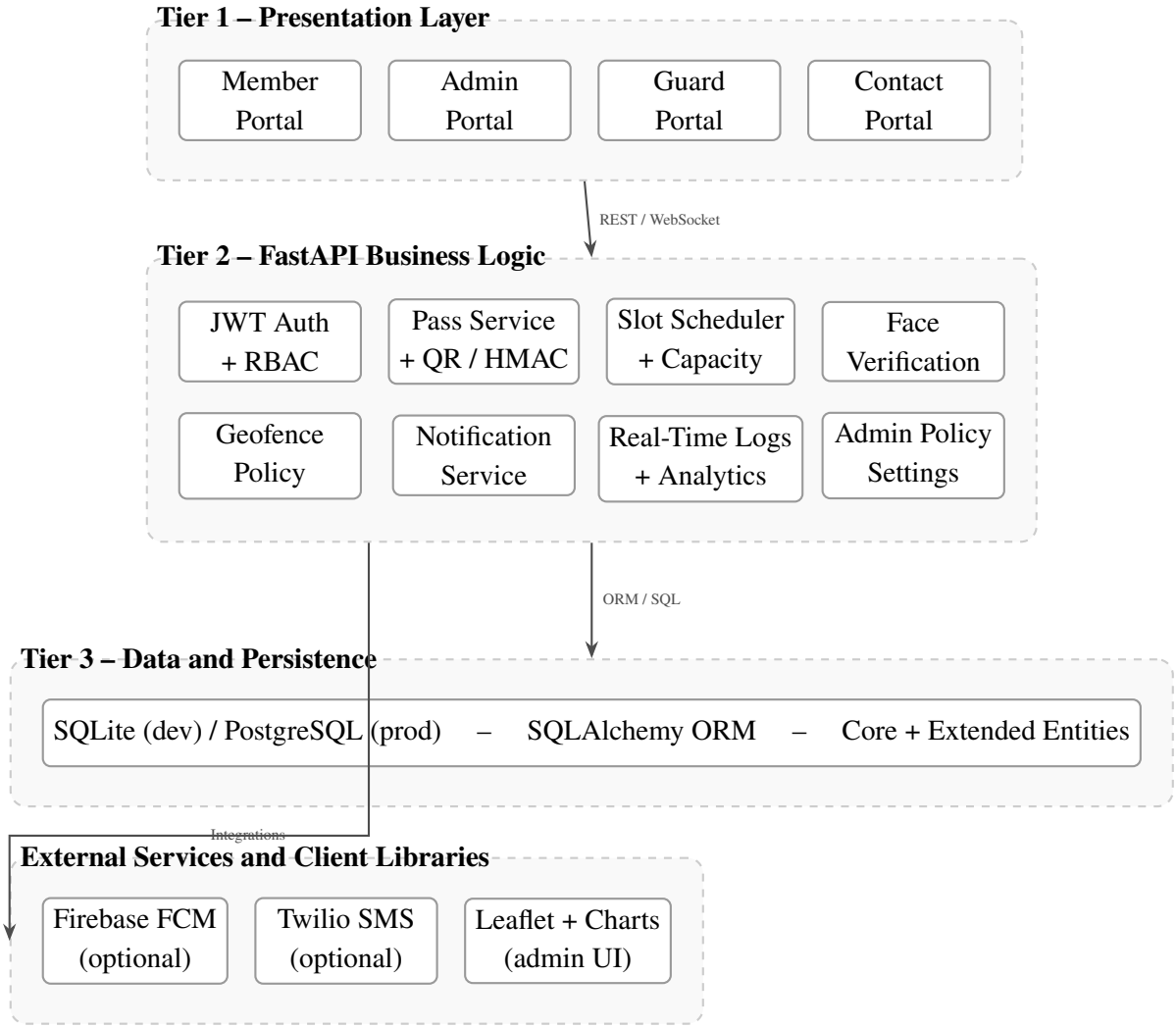
In-scope revision goals:

- redesign the report around distinct use cases,
- add slot-based access scheduling,
- bind selected passes to slot windows and capacity constraints,
- strengthen administrative analytics,
- add formal testing, requirements, and UML artifacts,
- align the deployed build, the report, and the demo script.

Out-of-scope for this milestone:

- native mobile applications,
- Kubernetes-scale deployment,
- anomaly-detection machine learning,
- RFID or NFC hardware integration.

Revised System Architecture



Current persisted entities:

- `users`
- `registration_requests`
- `passes`
- `scan_logs`

Proposed course-level extensions:

- `slots` – time window, gate, capacity, policy flags
- `slot_bookings` – user-slot reservation and status history

Actors and Responsibilities

Actor	Interface	Responsibility
Member	Member Portal	Request access, reserve slots, request standard passes, generate instant on-campus passes, maintain profile, trigger emergency exit
Admin	Admin Portal	Approve accounts and passes, define slot schedules, configure geofence and policy, monitor operations, review analytics
Guard	Guard Portal	Scan QR codes, verify face for restricted entries, enforce slot windows, log exceptions at the checkpoint
Contact	Contact Portal	View recent movement history through signed link access and receive alert subscriptions
Visitor	Admin-mediated flow	Receive one-time or scheduled guest access through host or admin provisioning

Reframed Use Cases

This revised set intentionally limits GPS-specific logic to only two use cases: UC06 and UC11. The remaining use cases focus on different actors, policies, and system goals.

UC01 – Access Request Submission

Primary actor: Member applicant

Goal: Request system access for authorized use.

Preconditions:

- The applicant does not yet have an active account.

Main flow:

1. The applicant opens the Member Portal and fills in identity and role details.
2. The system validates email, personnel ID, and request completeness.
3. A pending `registration_request` record is created.
4. Admin users are notified that a new account request is awaiting review.

Alternate flow:

- If the email or personnel ID already exists, the request is rejected immediately.
- If another pending request already exists, the system prevents duplicate submission.

Postconditions: A pending access request is stored and visible in the admin queue.

UC02 – Account Review and Provisioning

Primary actor: Admin

Goal: Approve or reject a pending account request and provision the correct role.

Preconditions:

- An admin is authenticated.

- At least one pending registration request exists.

Main flow:

1. The admin reviews submitted identity details and requested role.
2. The admin approves the request and optionally adjusts the final role.
3. The system creates the user record and activates login access.
4. The request record is updated with review metadata and linked to the created account.

Alternate flow:

- The admin rejects the request and records a review note.

Postconditions: The request is permanently marked approved or rejected, and the user account state is synchronized.

UC03 – Slot Definition and Capacity Planning

Primary actor: Admin

Goal: Define controlled entry or exit windows for members and visitors.

Preconditions:

- The admin is authenticated.

Main flow:

1. The admin creates a slot with label, gate, start time, end time, and maximum capacity.
2. The admin optionally enables policy flags such as GPS-required or face-check-required.
3. The slot is saved and published to eligible users.
4. The dashboard shows remaining capacity and slot status.

Alternate flow:

- Invalid time ranges or zero capacity are rejected at validation time.
- Overlapping administrative policies can be flagged for review.

Postconditions: A slot definition is available for booking and policy enforcement.

UC04 – Slot Booking by Member

Primary actor: Member

Goal: Reserve an approved access window before reaching the gate.

Preconditions:

- The member account is active.
- At least one eligible slot is available.

Main flow:

1. The member views available slots in the portal.
2. The member selects a slot matching the intended entry or exit time.

3. The system checks capacity, duplication, and role restrictions.
4. A `slot_booking` record is created and linked to the eventual pass flow.

Alternate flow:

- If the slot is full, the booking is rejected and the user is asked to choose another slot.
- If the member already holds a conflicting reservation, the duplicate booking is blocked.

Postconditions: The member has a reserved access window recorded in the system.

UC05 – Standard Pass Request with Admin Approval

Primary actor: Member

Goal: Request a non-urgent pass that still requires administrative approval.

Preconditions:

- The member is authenticated.

Main flow:

1. The member submits reason, pass type, and optional supporting data.
2. The system creates a pending pass request.
3. Admin users review the request and approve or reject it.
4. On approval, the system generates a time-limited HMAC-signed QR token.

Alternate flow:

- Rejection returns a final status and optional review feedback.

Postconditions: The pass is either rejected or converted into an approved QR-based credential.

UC06 – Instant On-Campus Pass Generation

Primary actor: Member

Goal: Generate an immediate pass from inside the configured campus boundary.

Preconditions:

- The member is authenticated.
- Browser geolocation is available.

Main flow:

1. The member requests an instant entry or exit pass.
2. The client captures latitude and longitude from the browser geolocation API.
3. The server validates distance to the configured campus boundary with the allowed GPS buffer.
4. If valid, the pass is auto-approved and the QR token is generated immediately.

Alternate flow:

- If the location is outside the boundary, the request is denied.
- If GPS permission is unavailable, the user falls back to the standard approval path.

Postconditions: A geofence-compliant instant pass is issued or the request is denied.

UC07 – Visitor Pass Creation

Primary actor: Admin

Goal: Issue one-time or scheduled access to an external guest.

Preconditions:

- The admin is authenticated.
- Visitor identity or host details are available.

Main flow:

1. The admin enters visitor details, validity period, and optional assigned slot.
2. The system creates a restricted pass with a short validity window.
3. A QR token is generated and shared to the visitor through the approved channel.
4. The guard sees the visitor flag during verification.

Alternate flow:

- If the requested window conflicts with policy or capacity, the visitor pass is not issued.

Postconditions: A visitor credential is issued with tighter policy controls than a normal member pass.

UC08 – QR Verification at the Checkpoint

Primary actor: Guard

Goal: Validate a presented QR pass before allowing entry or exit.

Preconditions:

- A guard is authenticated.
- The member or visitor presents a QR token.

Main flow:

1. The guard scans or pastes the QR token.
2. The system parses the token and recomputes the HMAC signature.
3. The system checks expiry, pass state, one-time-use status, and slot window if applicable.
4. On success, the pass is marked used and a scan log entry is created.

Alternate flow:

- Signature mismatch produces an invalid-token rejection.
- Expired or already-used passes are denied and logged.
- Slot-window mismatch is denied with an explanatory message.

Postconditions: The checkpoint outcome is committed to the audit log and reflected in dashboards.

UC09 – Restricted Access Face Verification

Primary actor: Guard

Goal: Confirm identity for entries that require stronger proof than QR alone.

Preconditions:

- The presented pass is valid.
- The access policy requires face verification or the guard manually enables it.

Main flow:

1. The guard captures a live face image after the QR is accepted.
2. The system extracts the live encoding and compares it with the enrolled template.
3. Confidence and match status are computed.
4. The guard is shown a clear allow or deny result with reason.

Alternate flow:

- If no face is enrolled, the system records the condition and triggers fallback handling.
- If the image is poor or no face is detected, the verification is retried or denied.

Postconditions: Identity-sensitive access decisions become auditable and policy-driven.

UC10 – Contact Subscription and History Review

Primary actor: Contact

Goal: View recent movement history and subscribe to future alerts without a full account.

Preconditions:

- The contact receives a signed access link from the member.

Main flow:

1. The contact opens the signed link.
2. The system validates the signed token and maps it to the correct member record.
3. The contact sees recent successful entry and exit history.
4. Optional phone number and notification token are saved for future alerts.

Alternate flow:

- Invalid or expired signed links are rejected.

Postconditions: The contact can monitor movement history without being provisioned as a full user.

UC11 – Live Monitoring and Policy Adjustment

Primary actor: Admin

Goal: Monitor operations in real time and refine access policy from the control dashboard.

Preconditions:

- The admin is authenticated.

Main flow:

1. The admin opens the dashboard and receives live WebSocket updates.
2. The system shows recent scans, hourly counts, daily trends, and error conditions.
3. The admin adjusts policy settings such as geofence center, radius, or slot availability.
4. Updated settings become active for future requests.

Alternate flow:

- If live WebSocket delivery is unavailable, the dashboard falls back to polling-based refresh.

Postconditions: Administrative awareness and policy control remain synchronized with gate activity.

UC12 – Emergency Exit Handling

Primary actor: Member

Goal: Exit the secured area immediately during an urgent situation.

Preconditions:

- The member is authenticated.

Main flow:

1. The member triggers an emergency exit request from the portal.
2. The system records an emergency scan log entry without waiting for pass approval.
3. Admin users are notified about the emergency event.
4. The event appears in real-time logs for oversight.

Alternate flow:

- Notification delivery failure does not cancel the emergency record or exit approval.

Postconditions: The emergency action is granted quickly while preserving a full audit trail.

Data Model Revision

Entity	Key Fields	Purpose
`users`	id, role, member ID, face fields, contact fields, validity	Identity, role, and notification ownership
`registration_requests`	requester details, requested role, review status, reviewer metadata	Access provisioning workflow
`passes`	user ID, reason, pass type, status, QR token, expiry, usage fields	Access credential lifecycle
`scan_logs`	pass ID, scanner ID, result, pass type, details, emergency	Immutable audit history
`slots` (new)	label, gate, start/end time, capacity, active, policy flags	Time-window and capacity definition
`slot_bookings` (new)	slot ID, user ID, pass ID, booking status, booked at	Reservation and allocation history

Recommended pass extensions for the next implementation cycle:

- `passes.slot_id` to bind a pass to a reserved slot,
- `passes.access_mode` to distinguish standard, instant, slot-bound, and visitor flows,
- `passes.requires_face_check` where policy requires stronger verification.

Technology Stack

Technology	Version	Role in the Revised Project
Python	3.11+	Core backend language
FastAPI	0.115.x	REST API, dependency injection, request validation, async endpoints
Uvicorn	0.30.x	ASGI server for HTTP and WebSocket transport
SQLAlchemy	2.0.x	ORM and persistence layer across SQLite and PostgreSQL
Pydantic + pydantic-settings	2.x	Schema validation and environment-based configuration
PyJWT	2.9.x	JWT authentication and signed contact-link tokens
Passlib + bcrypt	1.7.x	Password hashing and verification
HMAC-SHA256	stdlib	Tamper-proof QR signature generation and verification
OpenCV	4.8.x	Default lightweight image processing and face feature extraction
face-recognition	optional	Alternate heavier face backend when explicitly enabled
Pillow	10.x	Image validation and normalization
geopy	2.4.x	Geodesic distance checks for campus boundary validation
Shapely	2.0.x	Polygon-based geofence support
firebase-admin	6.3.x	Optional server-side push notifications
Twilio	8.10.x	Optional SMS delivery
APScheduler	3.10.x	Background scheduling and future analytics jobs
jsQR	CDN	In-browser QR scanning in the Guard Portal
QRCode.js	CDN	Client-side QR rendering
Chart.js	CDN	Dashboard charts for analytics and trends
Leaflet.js	CDN	Interactive map for geofence and policy setup

Deployed Baseline vs Course-Level Target

Dimension	Current Deployed Baseline	Course-Level Target
Use-case framing	Feature-driven and partly repetitive in the earlier report	Actor-centered, 12 distinct use cases
Access decision	QR, approval state, optional face, geofence-aware instant pass	Policy-driven QR + slot + face + time-window enforcement
Scheduling	No explicit slot reservation	Slot creation, booking, capacity, and time-window validation
Analytics	Live logs and dashboard counts	Historical trend, compliance rate, slot utilization, export endpoints
Documentation	Demo report exists	Final report, UML, requirements, test plan, polished demo script
Testing	Minimal ad hoc verification	Structured unit and integration test coverage

Implementation Roadmap

Phase 1 – Documentation and Scope Alignment

- finalize the revised system story and actor-goal use cases,
- update terminology so the report, UI, and backend role names stay consistent,
- prepare UML and requirements around the revised scope.

Phase 2 – Slot-Based Access Scheduling

- add `slots` and `slot_bookings` schema support,
- build admin slot creation and capacity management,
- add member slot booking flow,
- enforce slot window checks during guard verification.

Phase 3 – Analytics, Testing, and Hardening

- add analytics endpoints for peak hours, trend lines, and compliance metrics,
- create pytest coverage for crypto, auth, geofence, and slot logic,
- harden production settings such as CORS and event-loop handling,
- align the deployed demo with the revised report and final presentation flow.

Immediate Two-Day Priority Plan

Day	Priority Deliverables
Day 1	finalize revised report, lock the 12 use cases, define slot schema and API contracts, prepare admin and member UI changes
Day 2	implement slot creation and booking, bind passes to slot windows, polish analytics views, update demo flow, redeploy

Testing and Evaluation Plan

Core automated tests to add:

- `test_crypto.py`: token generation, tamper detection, expiry behavior, replay protection
- `test_auth.py`: valid login, invalid password, inactive user, role enforcement
- `test_geofence.py`: inside, outside, invalid coordinates, buffer zone, policy-disabled mode
- `test_slot_scheduler.py`: capacity limit, double booking, expired slot, slot-bound verification
- `test_face_auth.py`: enrollment status, match, mismatch, degraded image handling

Evaluation metrics for the final project:

- QR verification success rate,
- average checkpoint verification time,
- percentage of denied replay attempts,
- slot utilization percentage,

- geofence compliance rate for instant passes,
- alert-delivery success ratio when notification providers are configured.

Professional Submission Positioning

For the final course submission, SecureGate should be presented as:

- a **deployed baseline system** that already demonstrates real access-control flows,
- a **course-level revision** that introduces slot-based policy enforcement,
- a **distributed systems project** with multi-portal coordination, real-time logs, cryptographic verification, and configurable operational policy.

This positioning is stronger than simply claiming many isolated technical features, because it demonstrates:

- problem decomposition,
- policy-aware system design,
- realistic incremental engineering,
- honest separation of implemented and planned work,
- a coherent path from prototype to course-quality system.

Conclusion

The revised SecureGate report turns the project into a cleaner, more defensible course submission. Instead of repeating GPS-heavy or implementation-only use cases, it now emphasizes distinct user goals, policy-driven access control, and a concrete systems extension through slot scheduling. This makes the next development steps clear:

1. keep the deployed baseline stable,
2. implement slot-based scheduling and enforcement,
3. strengthen analytics, tests, and formal project documentation,
4. present the final project as a structured access-control platform rather than only a gate-pass demo.